

线性方程与最小二乘几何实验

实验目标与先修

本实验把手算、几何条件和浮点结果放在同一份可重跑记录里。你将重建一个精确最小二乘答案，比较 `lstsq`、库 QR、教学 Gram-Schmidt、正规方程 `solve` 和显式求逆，再观察秩亏、近共线和不同方向扰动的影响。

直接先修第十讲，并通过它继承向量内积、矩阵形状与 NumPy 基础。不需要微积分或 SVD 理论。NumPy 某些库接口内部使用成熟分解，包括 SVD；本讲按官方接口使用和记录结果，其几何理论在第十二讲补充。不要把暂时使用一个可靠实现说成已经学完它的证明。

所有数据是确定性合成小矩阵，无随机抽样、无 GPU、无网络调用、无付费服务。建议分三次完成：手算与证明；基础浮点核验；病态与边界实验。实际运行很快，主要时间应花在解释结果上。

一 文件与环境

保留完整的 011 文件夹。`lecture.md` 是正文，`answers.md` 是十五题详解，`experiment.py` 是核心脚本，`experiment.ipynb` 是分步骤实验本。`data/main.csv` 包含三条主例记录；其他矩阵在脚本里按明确公式构造，不依赖隐藏文件。

制作核验版本为 Python 3.12.14、NumPy 2.3.5，计算数据类型为 `float64`。核心实验只需要 NumPy；讲义插图已附带，不要求学生运行作图工具。若想重新生成插图，还需要 Matplotlib，但这不是完成数学验收的前提。

已有 Anaconda 时，在 Navigator 中启动 Jupyter Notebook 或 JupyterLab，从文件列表进入 011 文件夹，打开 `experiment.ipynb`，选择 Python 3 内核。重启内核后 Run All，确保从空状态开始。逐格执行可以用 Shift+Enter，但第一次核验应从头执行全部单元。

若环境无法导入 NumPy，先检查是否选中了装有 NumPy 的 Anaconda 环境，而不是另一个空 Python。需要安装时，可在自己的环境中用 `python -m pip install numpy==2.3.5` 安装指定版本；使用学校或组织管理电脑时遵守其软件安装规则。Jupyter 是打开实验本的界面，不是本讲计算公式的一部分。

也可以在此文件夹打开终端或 Anaconda Prompt，运行 `python experiment.py`。脚本会在 `outputs` 内生成 `experiment_report.json` 和 `method_comparison.csv`；它不会修改原始 CSV。重跑会更新这两个生成文件。若保存探索版本，先复制整个文件夹。

二 主例纸笔核验

从 `data/main.csv` 读出输入 0、1、2，观察 1、2、2。构造第一列为 1、第二列为输入的 3×2 矩阵 A。不要把参数顺序换成斜率在前却仍沿用截距在前的参考答案。

先证明精确方程无解。前两条要求截距为 1、斜率为 1，第三条不满足；或者完成增广矩阵消元，得到最后一行 $0 = -1$ 。记录 A 的精确秩为 2，增广矩阵精确秩为 3。

随后手算正规方程： $A^T A$ 的四个元素依次为 3、3、3、5， $A^T y = (5, 6)^T$ 。两式相减求斜率 $1/2$ ，再求截距 $7/6$ 。预测为 $7/6, 5/3, 13/6$ ；本讲残差固定为观测减预测，得到 $-1/6, 1/3, -1/6$ ，SSE 为 $1/6$ 。

最后计算残差与两列各自的内积，均应为 0。不要只算 SSE 就结束：正交关系是最优性的结构证据。程序的 Fraction 单元会用精确有理数重现这些值；后面的 NumPy 路线才进入浮点近似。

三 从投影手算 QR

第一列长度为 $\sqrt{3}$ ，单位方向是 $(1, 1, 1)^T / \sqrt{3}$ 。第二列沿它的投影系数为 $\sqrt{3}$ ，投影向量是 $(1, 1, 1)^T$ 。扣掉后剩余 $(-1, 0, 1)^T$ ，长度为 $\sqrt{2}$ ，单位化得到第二个 q 。

写出 Q 与 R，手算 $Q^T Q = I_2$ 、 $QR=A$ ，并算 $Q^T y = (5/\sqrt{3}, 1/\sqrt{2})^T$ 。从 R 的最后一行回代，再回到第一行，得到同一组参数。请在每一步旁边写形状：Q 为 3×2 ，R 为 2×2 ， $Q^T y$ 为二分量。

Notebook 会比较库 QR 与教学修改 Gram-Schmidt。库可能选择不同的列符号，正确标准是重构、正交和最终预测，而不是逐元素与手写 Q 完全相同。报告保留重构缺陷的程序度量以及正交缺陷；注意程序对矩阵 `np.linalg.norm` 未指定 `ord` 时使用 Frobenius 范数，下面会明确解释。

这里为避免混用记号，应将程序中的重构与正交矩阵误差读作 Frobenius 范数：先把全部元素平方相加再开平方。向量残差与参数差则使用欧氏二范数。正文的条件数采用算子二范数，三个地方不是一个默认接口含义。

四 逐方法比较时到底比较什么

核心报告记录七个方法名。`lstsq` 直接处理最小二乘；`qr_library` 用库 QR 后求解上三角系统；`qr_teaching_mgs` 用教学修改 Gram-Schmidt 后回代；`normal_solve` 显式形成 G 与 h，再解 $G\beta = h$ ；`normal_inverse` 形成 $G^{-1}h$ ；`solve_A` 和 `inverse_A` 直接作用原矩阵，只在方阵条件合适时合法。

主例 A 不是方阵，因此最后两个直接方阵路线预期失败，报告保留异常类型。这不是最小二乘任务无法完成，而是对应接口用途不匹配。其余五条主例路线应在绝对与相对容差各 10^{-11} 内符合精确参考。

每个成功方法记录系数、参数差、预测差、原始残差、SSE 与 $A^T r$ 的长度。参数差是与当前案例明示参考比较：主例参考是已证明的最优解，近共线无噪声例参考是生成参数。失败则记录状态、异常类型和说明，不能填一个伪造的零误差。

比较脚本没有断言“显式求逆一定最差”或“每个稳定路线都必须胜过正规方程”。这类逐例排名不是一般定理。你应保留实测结果，并从条件数、构造方式和舍入解释它。不同 BLAS/LAPACK 版本的末尾几位和个别临界失败可能不同，应注明环境。

五 重复列与最小范数

把输入列复制一次，构造 D 的三列为常数、 x 、 x 。 D 有三个参数却只有两个独立列方向。报告会给出 `lstsq` 选择的最小范数参数，约为 $7/6, 1/4, 1/4$ 。

纸上另外取 t 为 -2 、 0 、 3 ，形成参数 $(7/6, 1/4 + t, 1/4 - t)^T$ 。逐项核对预测完全相同，SSE 也相同；参数长度却在 t 为 0 时较小。一般的最小范数结论来自讲义中的平方展开，不是因为只比较三项就证明了任意 t 。

此例还检查 `lstsq` 返回的 `residuals` 数组。它可能是空数组，即使原始残差并不为零。请始终自行计算 $y - D @ \text{beta}$ 。报告中会保留返回值与重算的 SSE，避免把接口占位约定当成数学结论。

六 从满秩逐渐走近不可辨别

近共线设计的第一列全为 1 ，第二列为 $1 + \varepsilon t$ ，其中 t 是 -2 、 -1 、 0 、 1 、 2 。取真实系数 1 与 2 ，令 y 由 A 乘该系数得到。依次设置 $\varepsilon = 2^{-k}$ ， k 为 10 、 20 、 26 、 30 、 40 、 50 。

精确公式只要 ε 非零就有秩 2 。脚本还把实际存储的浮点数逐个转换为精确 `Fraction`，用小规模行消元确认这六个存储矩阵的精确秩也是 2 。同时记录 NumPy 默认数值秩；最后一个案例可能被默认阈值判为 1 。这两个答案描述的条件不同，不相互矛盾。

在本次制作环境， k 为 10 、 20 、 26 时 `normal_solve` 返回的系数误差恰好为 0 ； k 为 30 、 40 、 50 时 `normal_solve` 与 `normal_inverse` 对形成的 G 报奇异错误。直接 `lstsq` 在 k 为 40 时原始残差仍约为 10^{-15} ，系数误差却约为 4.22×10^{-4} ； k 为 50 时默认阈值保留一个方向，系数约为 1.5 、 1.5 。

这些是实测案例描述，不应写成每台机器都必须逐位重复的定理。它们展示两件事：正规矩阵可能先于原设计丢失区分方向的能力；很小残差不保证参数准确。报告中的 `condition_gram_computed` 是已形成的浮点 G 的诊断，不是理论精确 G 的条件数。极度病态时，二者可能显著偏离。

教学 QR 会检查剩余方向或 R 对角元是否过小。本程序阈值为机器精度乘最大形状维数再乘 A 的算子二范数。这是本教学回路路线的明确拒绝策略，不是声称与 `lstsq` 的 SVD 阈值判定完全等价。被拒绝时，应换适合有效秩的成熟求解途径，而非删除保护继续除以极小数。

七 改变阈值时保留任务解释

固定 k 为 40 ，分别运行 `rcond=None`、 10^{-14} 、 10^{-12} 、 10^{-10} 。本次结果的有效秩依次为 2 、 2 、 1 、 1 。每次都保存系数、重算残差及参数差。

`None` 使用 NumPy 默认规则；显式 `rcond` 是与最大奇异值比较的相对截止比例，不是直接对每个矩阵元素的绝对阈值。具体奇异值理论在下一讲学习，目前先按官方接口记录并理解“保留或丢弃弱方向”的效果。

提高阈值往往使两个系数更接近平均分配，但这改变了有效问题，不证明真实生成系数就是平均值。不要以“结果看起来平滑”作为选择阈值的唯一理由。真实数据里还要考虑测量精度与希望回答的问题。

八 用两种扰动拆开参数和预测

固定 $\varepsilon = 2^{-20}$ ，取 $\eta = 2^{-30}$ 。第一种扰动是 $\Delta y = \eta t$ ，位于列空间中。精确系数应改变 $(-\eta/\varepsilon, \eta/\varepsilon)$ ，即 $(-2^{-10}, 2^{-10})$ 。它们约为千分之一，而观察扰动只有十亿分之一量级。

第二种扰动是 $\eta(1, -2, 1, 0, 0)^T$ 。它与常数列和 t 都正交，精确拟合应不变，扰动留在残差里。先手算这两个内积为零，再读浮点报告；近似零的计算偏差不能直接解释成真实机制。

本次第一种数据变化范数约为 2.9451×10^{-9} ，参数变化约为 1.3811×10^{-3} ，预测变化仍约为 2.9451×10^{-9} 。第二种的预测变化只在约 10^{-15} 的舍入尺度，新的残差接近加入的扰动大小。

观察空间内的投影不放大绝对扰动，不意味着参数绝对稳定，也不意味着离开当前特征结构的新点预测稳定。请把三类数字分别报告，不只选一个最好看的误差。

九 边界与形状检查

自动检查会拒绝一维 A 、二维列状 y 、行数不匹配、空行或空列设计以及 NaN。这里拒绝二维列状 y 是本课程包装接口的选择，为了避免误读广播；NumPy 的原生 `lstsq` 可以处理多个右端，不能把我们的窄接口限制误称为库不支持。

有效边界还包括一个方程一个参数、单列均值、宽矩阵的最小范数解以及全零设计。全零设计配 y 为 1、2、3 时，最小范数参数为零，但 SSE 为 14；返回 `residuals` 仍可能为空。这是识别接口语义的最小反例。

自动测试另核验 QR 形状、重构和正交，精确增广秩，行缩放改变最小二乘，以及对角矩阵条件数平方关系。共 14 项边界组，每一组有具体目的。通过有限检查不等于证明一般算法稳定性，完整数学结论仍来自讲义里的假设与推导。

十 排错与交付

找不到模块或数据时，核对完整目录与 Notebook 位置；版本不同则先记录环境并核对容差。修改数据后固定主例断言可能按预期失败，不能直接删除 `assert`。CSV 中 `failed` 可能是安排好的形状或奇异案例，应查对应解释。

Notebook 最后应打印“脚本与逐步核对一致：True”。遇到未处理错误后须重启内核并 Run All，不能沿用旧输出。提交 Fraction 手算、正交最优性证明、QR 手算、方法比较、秩阈值说明和两种扰动报告；分别回答可达预测、参数唯一性、扰动敏感性、计算误差与证据范围。

制作时已在新 Python 进程的真实 ipykernel 进程内内核顺序执行。未测试浏览器 Jupyter UI、外进程套接字传输及本机 Anaconda 安装；环境和执行范围另见 README 与 test-result.json。